Iscandar Revisited: Expansion to k=4 Theory For Running Debian

Andrew Jonathan Fine

November 2026

Abstract

This paper extends a deterministic memory allocator from radix depth k=3 to k=4 and tests the extension against a real operating system load. The k=3 design, published previously, allocates from a single cube of 32,768 sectors using an exact-fit bijection at radix 32. The k=4 extension composes cubes into beams under a rotary allocation rule, preserving the invariants of the k=3 theory: exact fit, bounded probes, and placement as a pure function of the request stream.

The load is a complete allocation census of an embedded Debian boot, captured from the kernel's own tracepoints: 812,106 allocations, with a standing set of 56,023 live objects at peak. That census rules out the k=3 design unconditionally, since one cube mints at most 32,768 handles. Replaying the identical stream through a four-cube beam at a 4 KiB grain, the k=4 system carries the entire load with no refusals and no free-path anomalies, peaking at 56,535 sectors against 56,599 pages granted by a binary buddy control in a matched arena. The 64-page difference is the buddy's power-of-two rounding, which exact fit holds instead as spendable free sectors. At a tighter 256 MiB arena the beam still refuses nothing, though the buddy's largest remaining fit exceeds the beam's, a structural consequence of the beam's cube boundaries. The claim is support by replay, with a potential DRAM savings, not in-kernel replacement.

A deterministic allocator can bear a general operating system's real load and gives nothing it cannot afford. Iscandar is able to conserve memory through implicit recognition of trapped and adjacent sectors.

To Jennifer: Everything. Always. Forever.

1 The Problem

The prior paper [1] established a deterministic allocator at radix depth k=3: a single cube of 32,768 sectors, exact fit by bijection, every request resolved by examining a fixed grid of counters in a fixed order, its costs bounded by ceilings that are theorems about the construction. Depth extension was named there as the route to scale. What the prior paper did not establish is whether the construction survives contact with a real operating system.

A cube mints at most 32,768 handles. A complete census of an embedded Debian boot, taken from the kernel's own tracepoints, shows a standing set of 56,023 live objects. No sector size cures a handle census: at k=3 the system cannot stand in for the operating system's allocator at any grain. The question is therefore not one of tuning but of structure.

We extend the construction to k=4, composing cubes into beams under a rotary rule, so the k=3 theory is preserved: exact fit, bounded probes, placement as a pure function of the request stream, and failure returned as a status. Then, we determine by replay if the extended system can bear the operating system's own measured load in the buddy allocator's place, judged in matched arenas by granted memory and by largest possible fit remaining (LPFR), at the end of the stream and at the instant of peak stress.

2 The Extension to k=4

2.1 Coordinates

The k=3 theory names every sector of a cube by a coordinate (plane, row, sector) under a bijection B_3 onto the displacements 0 to 32,767. The extension adds one field. A k=4 coordinate is a pair (c, x), where x is a k=3 coordinate and c, the cube index, ranges over the beam's cubes. The composed bijection is $B_4(c, x) = c \cdot 32768 + B_3(x)$. Nothing in the cube changes: its engines, its modes, and its proofs are the k=3 theory operating unchanged on x. The cube index is carried outermost, so projection back to k=3 is deletion of the leading field, and every k=4 name corresponds to exactly one sector of exactly one cube. The naming is a bijection because it is a composition of bijections.

2.2 The Pointer

Coordinates extend by adding a field; pointers extend by widening the packing. The k=4 pointer is a 64-bit word: one virtual bit, one reserved bit, a 30-bit start field composed of the 15-bit cube index over the 15-bit local start, and a 15-bit count field, the count stored reduced by one as at k=3. Forty-seven bits are in use. The seventeen spare bits sit in the start stack, above the cube field, where a further 15-bit start field would extend the same word to k=5; the count does not widen at any depth, because an allocation lives inside one cube by construction. A pointer is born virtual, minted only from a coordinate and a count; a pre-existing word enters only by adoption, under the k=3 discipline.

2.3 The Beam

A beam is a set of cubes, from 1 to 32, under a rotary order. Allocation first checks the beam's summed counters against the request, then asks the lead cube; on refusal it rotates and asks the next, and the first acceptance along the rotary order seats the allocation. Counts are checked before any commitment is made. At the moment of acceptance, and only then, the accepting cube's index is known, and the returned k=3 pointer is transformed: its coordinate acquires the cube index, and the pair is minted as a k=4 pointer. Deallocation is the allocation process read in reverse, as the bijection requires: the pointer decomposes into its cube index and its k=3 coordinate, the local pointer is unmarked in its own cube, and the beam's counter is decremented only on a success status.

2.4 What Is Preserved

The extension surrenders none of the k=3 guarantees. Fit remains exact: no request is rounded to a size class. Probes remain bounded: at most one probe per cube in the beam, each probe bounded by the k=3 ceilings. Placement remains a pure function of the request stream: the rotary order is fixed in advance, so the same stream produces the same pointers on every run. Live pointers never overlay, by construction, at either level. Failure is returned as a status, never as a crash or a silent substitution. And the arithmetic at every level is pure sector arithmetic: the sector size, the grain, is imposed only at the top, by whoever maps sectors to memory.

3 Desk Check

The theory section asserts; this section traces. We follow the code of `iscandar-k4` once, slowly, on a beam of four cubes, 131,072 sectors, and check at each step that the code does what the construction says it does. The wider design space of dynamic storage allocation is surveyed by Wilson et al. [4].

3.1 The Gates

A request is refused by arithmetic before it is refused by search. The beam first checks itself:

```
if (self.sectors_supported - self.count_overall) < request:
  return
```

and each cube, when asked, checks itself the same way inside `scan`, returning nothing without ever calling its engine if the request exceeds its own remaining count. Counts are checked before any commitment is made; a beam that cannot seat a request in sum never begins, and a cube that cannot seat it alone is never disturbed.

3.2 Rotation and First Acceptance

The allocate loop walks the rotary order: `scan` the lead cube; on an `MPUPointer` result, add the request to `count_overall` and transform; on refusal, `rotate()` and ask the next, at most once per cube. First acceptance along the order seats the allocation, and the loop bound is the cardinality itself: the probes are bounded by the width of the beam.

3.3 The Transform

The accepting cube's index is known only at acceptance, and the code takes it there:

```
coordinate      = pointer.coordinate () # MPUPointer
beam_coordinate = BeamCoordinate (cube, coordinate.sector,
                                  coordinate.row, coordinate.plane)
return CPUPointer (beam_coordinate, pointer.count)
```

Inside `CPUPointer`, the start field is computed as:

```
coordinate.cube * Cube.SECTORS_PER_CUBE + local.displacement,
```

which is the composed bijection of the theory section written as one line of arithmetic: $B_4(c, x) = c \cdot 32768 + B_3(x)$, with the local displacement standing for $B_3(x)$.

3.4 The Free Path

Deallocation decomposes the pointer, reloads the local k=3 pointer by:

```
MPUPointer.load_coordinate (coordinate.k3 (), pointer.count),
```

and unmarks it in the cube named by the pointer's own cube field. The beam's counter is decremented only when the returned status is `GEAR_SUCCESS`; the status itself is returned to the caller either way.

3.5 A Worked Trace

On a fresh `Beam(4)`: `allocate(100)` seats in cube 0 and returns the word `0x8000000000000063`: virtual bit set, start 0, stored count 99, the true count less one. `allocate(32768)` cannot seat in cube 0, which already holds 100 sectors and so fails its own per-cube gate; the beam rotates, cube 1 accepts, and the word is `0x8000000040007FFF`: start 32768, that is, cube 1 at local displacement 0, stored count 32767 for the whole cube. The beam's counter stands at 32,868. Freeing both pointers returns status 8, `GEAR_SUCCESS`, both times, and the counter returns to exactly 0. Nothing was searched that arithmetic had already refused, nothing was committed before its count was checked, and the two words decode, field by field, to the coordinates the theory predicts.

4 The Test

4.1 The Intention

The test as first conceived was a test of the published system, at k=3. A stock embedded Debian would be built and booted, every allocation it made under its own buddy allocator would be recorded, and the resulting stream would be replayed through the k=3 engine to learn whether one cube could bear a real operating system's boot. The expectation that the attempt was reasonable came from the uncrating: the original drive suggested an initramfs on the order of 128K under the buddy system, a modest load to set against 32,768 handles.

The apparatus was arranged to keep the load honest. ELBE was installed from its released packages and pinned, and the test image, Fubuki, was built from our project description alone: the tool is the stock car, and we authored no part of the Debian it builds. The image runs under QEMU on a serial console, with no video, and the boot ends at the login prompt: the prompt is the boundary of the experiment, and everything before it is the load. Recording is by the kernel's own built-in tracepoints at the kmalloc level. No source was marked up, patched, or otherwise persuaded: the list of allocations is the list the buddy system itself produced, in the order it produced it.

4.2 The Reality

The first boots failed, and they failed on the rig, not on Iscandar. At 128 MiB the image died in the initramfs unpack, stock Debian under its own allocator, before our system had any part in the proceedings, and the trace buffer sizing took attempts of its own. The configuration that boots honestly is 512 MiB, and it is at 512 MiB that the capture of record was taken: 1,650,424 events, the header counts equal, zero overruns, resolving to 812,106 allocations and their frees.

The census then answered the original question before any replay could. At peak, the boot holds 56,023 live objects. One cube mints at most 32,768 handles, at any sector size: the shortage is in handles, not in bytes, and no grain cures it. The k=3 system cannot stand in for this load, unconditionally. The test was therefore rebuilt around the k=4 extension, for both of its legs alike: the buddy control replayed over the same stream in matched arenas, and the experiment proper run through the beam.

Let the reframing be read correctly. It is not an indictment of Iscandar. The panics belonged to the test configuration, and the handle shortage marks the boundary of the domain the k=3 theory was published for: bounded loads, censused in advance. A general operating system boot stands outside that domain by census, and a design is not indicted by meeting a load it never claimed. What the measurement took from k=3, it gave to the question: not whether the published system suffices for Debian, but whether its extension does.

The test data was generated on the author's laptop. Fubuki was built with ELBE 15.9 from Debian 13 (trixie) and booted under QEMU on that machine; the trace used in this test is the kernel's own record of that boot.

5 The Control

The control is the incumbent, given the same load and every advantage of the matched configuration. The identical stream is replayed through a binary buddy allocator [2] over an arena of 131,072 pages of 4 KiB, the 512 MiB of the machine under test. Each request is first reduced to pages at the stated grain, then granted the smallest power of two page count that covers it, placed at the lowest address that fits, split and coalesced by the buddy rules. The model's list discipline is simpler than the kernel's; its geometry is the kernel's: powers of two, one arena, history-dependent placement. It is the geometry that is under test, and the geometry is not a straw man.

The harness discipline is shared with the experiment, and is stated here once. When the stream allocates through a pointer the driver still holds live, the old pointer is freed first and counted as an inferred free: 1,736 such events, the trace's own bookkeeping made explicit rather than silently orphaned. Frees of unknown pointers are counted and skipped: 83,968, the allocations for which the trace window holds no birth. Occupancy and largest possible fit remaining are sampled on the same schedule for both systems.

The control's calibration is exact. Its live set peaks at 56,023 objects occupying 56,535 pages of pure demand, reproducing the census of The Test to the page. What it grants is more than the demand, by the buddy's own arithmetic: peak granted 56,599 pages, 221.09 MiB, the excess of 64 pages being the power-of-two rounding of the live set's requests, a quantity fixed by the size distribution alone. At the end of the stream it grants 56,596 pages and refuses nothing at any point. Its largest possible fit remaining never falls below 65,536 pages: in an arena this size, against this load, a block of 256 MiB is grantable at every sampled instant.

That is the mark. The control carries the boot at a cost of 56,599 pages granted, with a vast contiguous remainder, in the arena the machine actually has. The experiment is asked to meet it in the same arena, at the same grain, under the same discipline.

6 The Experiment

The experiment is the same stream, the same discipline, and the same arena, with the beam in the buddy's place. A beam of four cubes stands over 131,072 sectors of 4 KiB, the 512 MiB of the machine under test. Each request is reduced to sectors on its requested bytes alone, and the driver keeps the live set itself, sampling every thousand events. Largest possible fit remaining is read off the engines' own sector stores, as the largest contiguous free run inside any one cube: by construction it cannot exceed one cube, 32,768 sectors, and that ceiling is part of the measurement, not an apology for it.

The beam carries the whole stream. Of 1,650,424 events it refuses nothing and reports no free-path anomaly: the 1,736 inferred frees and 83,968 unknown frees arise under the shared harness discipline stated in The Control. Its live set peaks at 56,535 sectors in 56,023 objects, reproducing the census of The Test exactly, by an independent route. At the end of the stream it holds 56,532 sectors in 56,020 objects. Its largest possible fit remaining never falls below 32,768 sectors: a whole cube is grantable at every sampled instant.

7 The Comparison

Against the control, the difference is one number with two readings. The control granted 56,599 pages at peak; the beam's demand at peak is 56,535 sectors, and under exact fit demand and grant are the same number. The 64 pages between them, 256 KiB, are the buddy's power-of-two rounding: memory the buddy locks inside its own grants, welded to the blocks that hold it, and the beam holds instead as free sectors, which answer requests. The difference is small at this arena and it is also absolute: there is a band of arena sizes 64 pages wide in which the two systems return different verdicts on this identical load, an arena of 56,560 pages sufficing for the beam and not for the control.

On contiguity the honest report is that this arena stresses neither system. The beam's largest possible fit remaining never falls below 32,768 sectors, and the control's never falls below 65,536 pages. Both floors are set by the arena's generosity rather than by either allocator's character under pressure, and this paper does not pretend otherwise: what the matched configuration establishes is parity on the incumbent's own axes, granted totals and remainder alike, from a system whose placements are a pure function of the stream.

Two figures accompany this section: occupancy over the stream for both systems, and largest possible fit remaining over the stream for both systems, The Real Debian Numbers: Macroscopically identical, microscopically different, and the microscopic difference is the contribution.

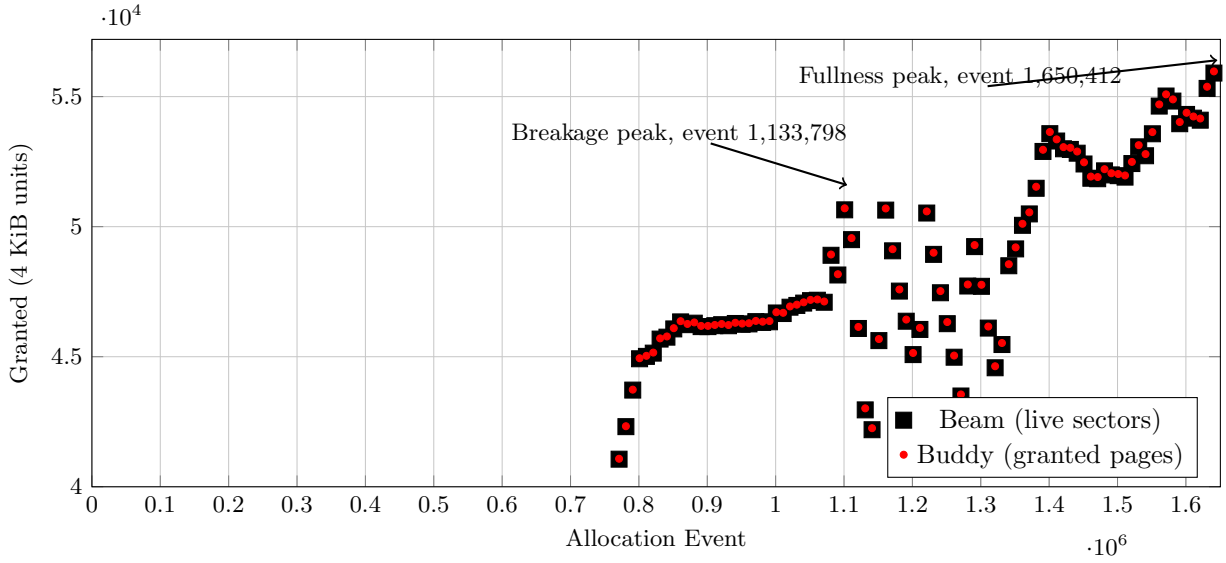


Figure 1: Occupancy over the stream, vertical axis magnified, every tenth sample: black squares, Iscandar; red rounds, buddy. The buddy rides above the beam by at most 64 pages. The arrows mark the two stress peaks examined in the Stress Testing section. The fullness peak, event 1,650,412, is where granted occupancy is maximal: the maximum stress maps are taken there. The breakage arrow marks the last sample before the fall: the breakage peak itself, event 1,133,798, lies at the bottom of the descent that follows, where the buddy's free space stood most fragmented in the halved arena: 24,543 pages free, its largest fit at its floor of 8,192 pages.

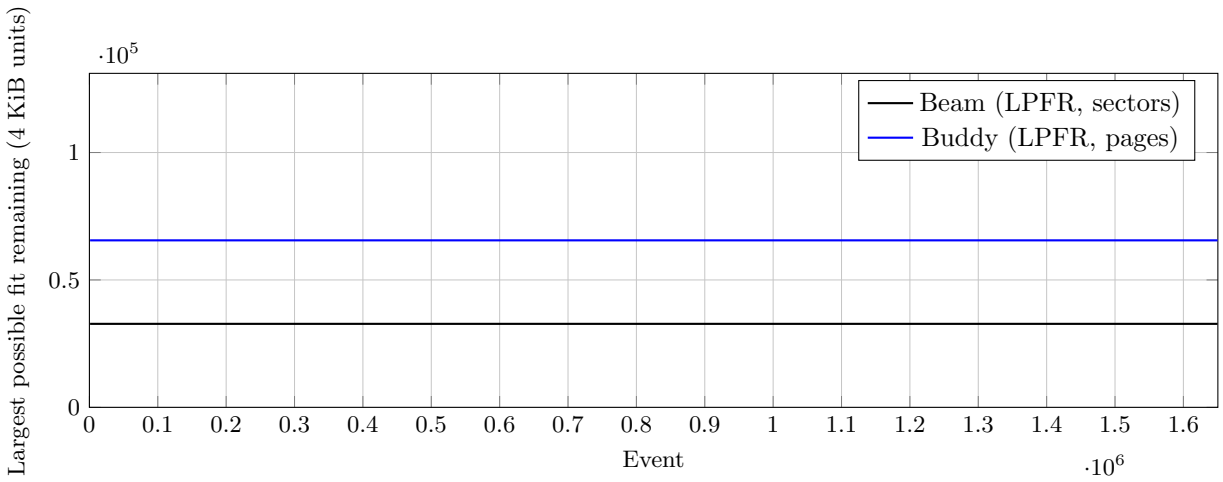


Figure 2: Largest possible fit remaining over the stream, in 4 KiB units. The arenas are matched at 131,072 units each, 512 MiB: four cubes for the beam, one span of pages for the buddy. Both lines are flat at every sample. The beam's line stands at 32,768 sectors, one whole cube, because no fit can cross a cube wall and at least one cube stands empty throughout. The buddy's line stands at 65,536 pages, half its arena in a single block. At this arena, neither system is stressed on contiguity.

A note on where the maximum falls. In both systems the instant of maximum stress arrives twelve events before the end of the stream, at event 1,650,412, and this is a property of the load, not of either allocator. A boot accumulates: most of what the kernel allocates is resident state, taken as its phase arrives and not freed before the prompt. Occupancy therefore climbs as a staircase: a long plateau near 20,500 live sectors, another near 46,300, then the final rise as the last services and the login stack come up. The churn rides on the staircase, its median lifetime seven microseconds, too small to outbid the accumulated set for the peak. And the end of the stream is the experiment's boundary, the login prompt, not a shutdown: the frees that would draw occupancy down were never recorded. Both systems are therefore judged at nearly the same instant for a structural reason: the machine finishes filling itself just before the trace closes, and the last dozen events, a few init paths freeing their scratch on the way out, lower the granted total by three pages, from 56,599 to 56,596.

Set side by side at the two instants of the fall, the systems trade the same ground in opposite shapes. At the crest the control stands nearly solid, 51,767 pages granted with its largest fit already at its floor of 8,192, while the beam holds 51,708 live sectors with its lead cube's largest run down to 15 sectors and its reserve cube offering 5,920. At the bottom the control holds 24,543 free pages but can offer no fit above 8,192, with 10,953 of those pages stranded in holes inside its span; the beam holds 24,602 free sectors, 59 more, its largest fit 5,377 in its emptier cube, and every free sector spendable at its own size, freed sectors having returned to service with no coalescing step between a free and its reuse. Neither system refused a request at either instant. The difference is in what their free memory is: the control's is a quantity, the beam's is an inventory.

8 Stress Testing

The matched configuration of The Control and The Experiment, a 512 MiB arena, stresses neither system: granted totals peak near 56,600 units of 131,072, and the largest possible fit remaining never falls below half the arena. To see the systems under pressure the arena is halved, to 256 MiB, and the stream is stopped at the instant of maximum stress: peak granted occupancy, which in both systems falls at event 1,650,412 of 1,650,424, twelve events before the end.

8.1 Buddy System

Within the control's run, the telling episode is a fall. Occupancy climbs its staircase to a crest at event 1,106,000, where 51,767 pages stand granted in the halved arena, then falls away as a phase of the boot lets its population die, bottoming at event 1,133,798 with 40,993 pages granted. Figure 3 maps the top of the crest, Figure 4 the bottom, and Figure 5 the end of the stream in the halved arena, where 56,596 pages stand granted of 65,536.

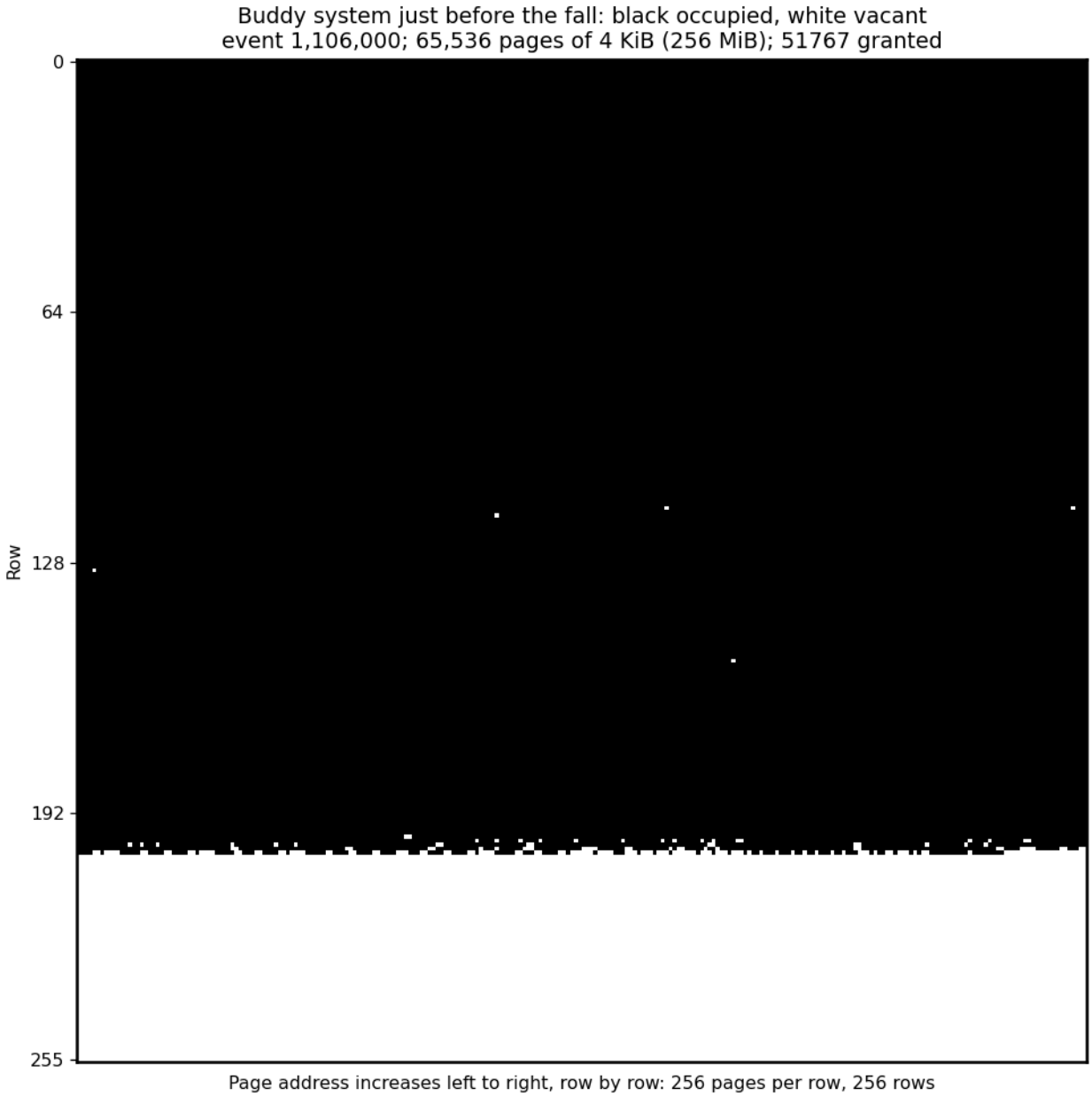


Figure 3: The buddy system at the top of the crest, 256 MiB arena, event 1,106,000. Black is occupied, white is vacant; each cell is one 4 KiB page. 51,767 pages are granted, and 180 vacant pages already stand inside the occupied span; the largest possible fit remaining is already at its floor of 8,192 pages.

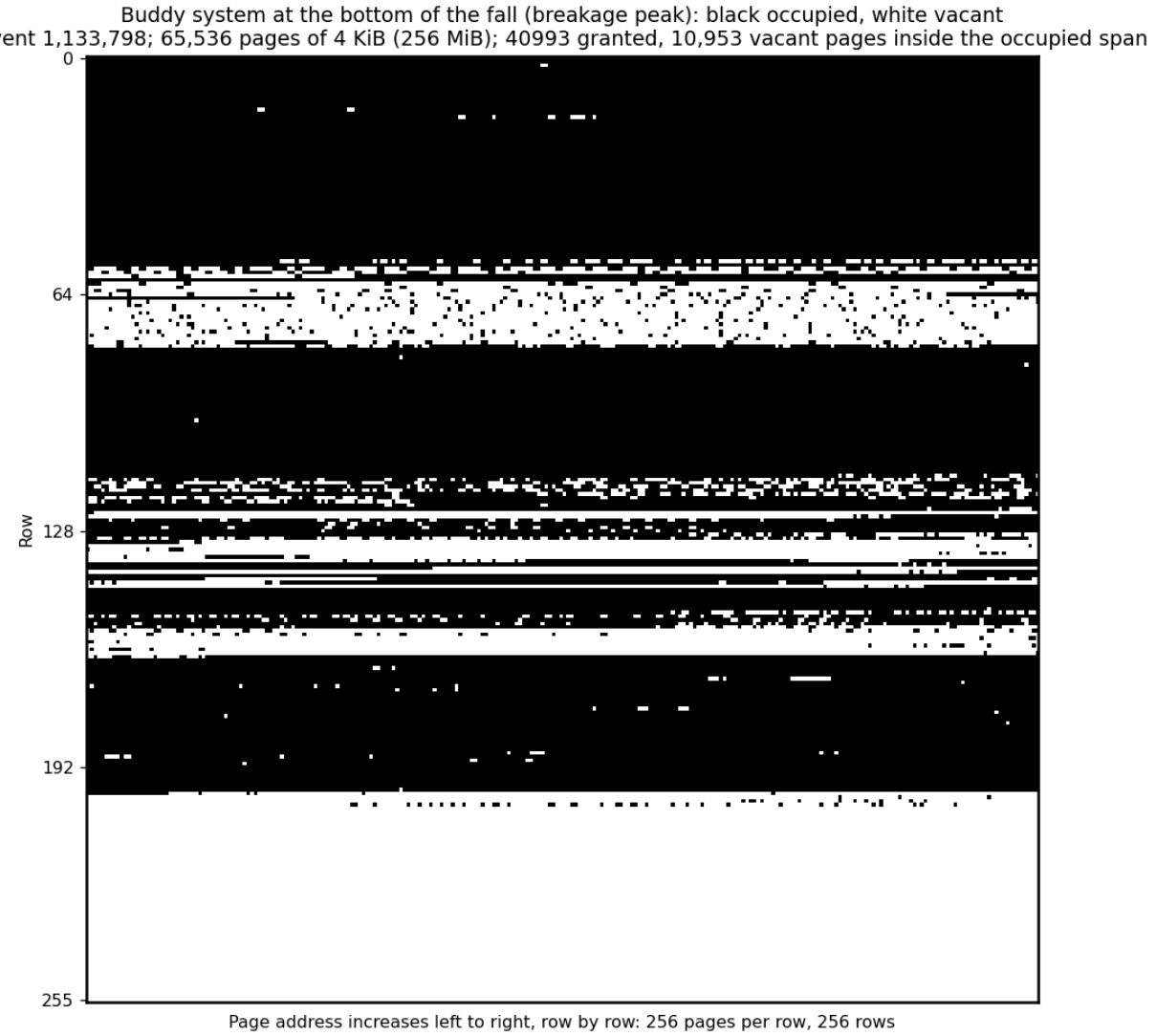


Figure 4: The buddy system at the bottom of the fall, the breakage peak, 256 MiB arena, event 1,133,798. Black is occupied, white is vacant; each cell is one 4 KiB page. Granted has fallen to 40,993, and the occupied span is hollow: 10,953 vacant pages in 1,108 separate holes, the largest hole 465 pages, with 24,543 pages free in all and no single fit larger than 8,192.

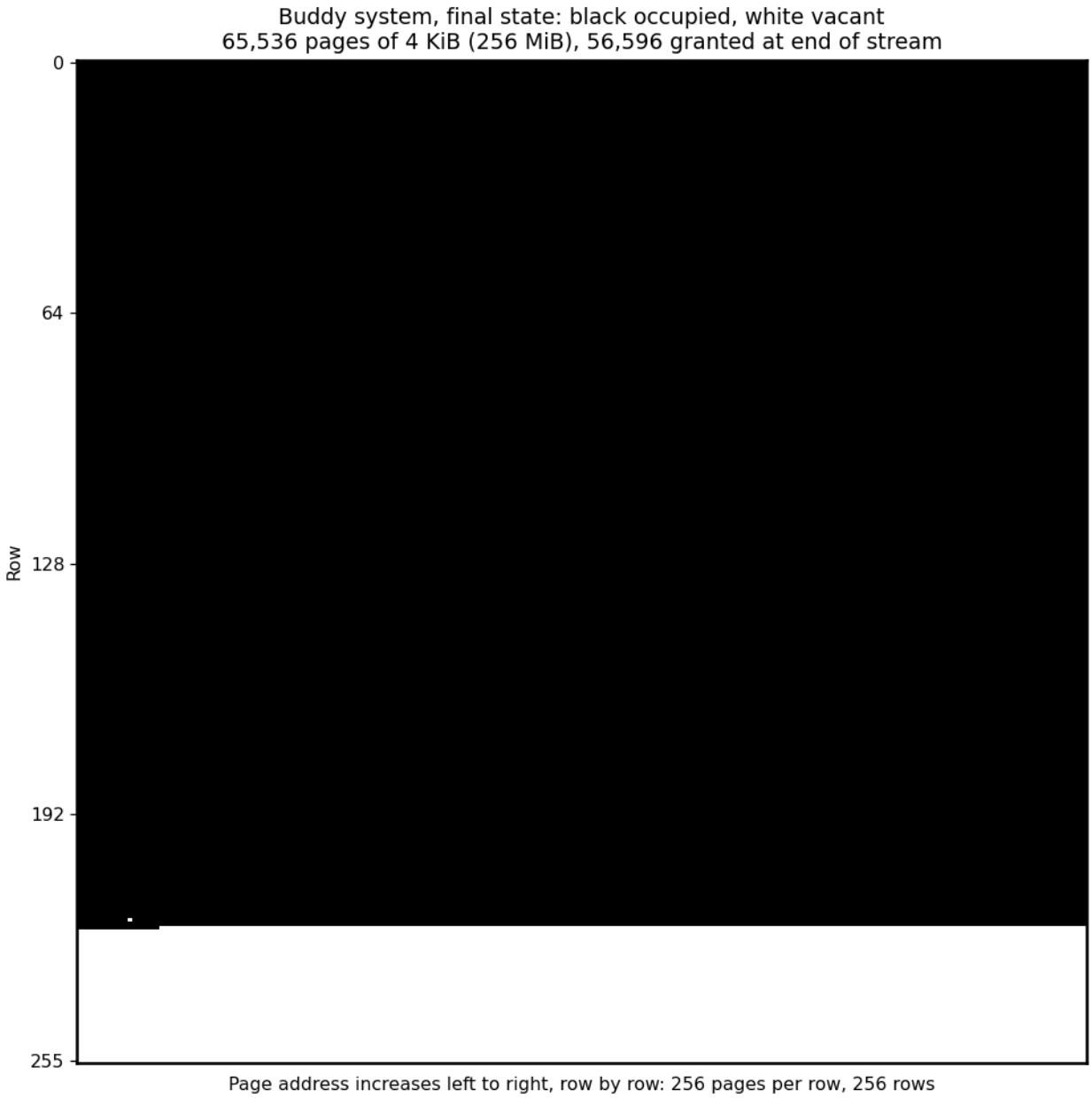


Figure 5: The buddy system at the end of the stream, 256 MiB arena. Black is occupied, white is vacant; each cell is one 4 KiB page. Inside the occupied span, which runs from page 0 to page 56,596, there is exactly one vacant page.

Read in order, the buddy system maps are the biography of the free space under first, best fit. At the crest the span is nearly solid: the climb has packed it, and only 180 holes stand inside it, though the largest fit is already at its floor. The fall hollows it: freeing punches a hole wherever a dying object lived, coalescing closes only the holes whose buddies are also free, and at the bottom the span holds 10,953 vacant pages it cannot offer as one piece. Then the climb out refills it from the bottom, first fit taking the lowest holes first, until at the end of the stream the span is solid again but for a single page. This shows from classic buddy system algorithm under first, best fit [3]: each allocation takes the lowest-addressed free block of the smallest sufficient order and splits it down, and each free coalesces with its buddy while the buddy is free. The system never refused a request on this load. But at the bottom of the fall it would have refused any request above 8,192 pages, with 24,543 pages standing free: coping in totals, barely coping in shape.

8.2 Beam

The beam is shown at the same two instants, in the same halved arena of two cubes. Figure 6 maps the top of the crest, event 1,106,000, where the beam holds 51,708 live sectors in 51,241 objects and cube 0, which has taken the climb, stands 97.5 percent full. Figure 7 maps the bottom of the fall, event 1,133,798, where the beam holds 40,934 live sectors in 40,480 objects. Figure 8 maps the end of the stream in the halved arena, where 56,532 sectors stand live and 9,004 are free.

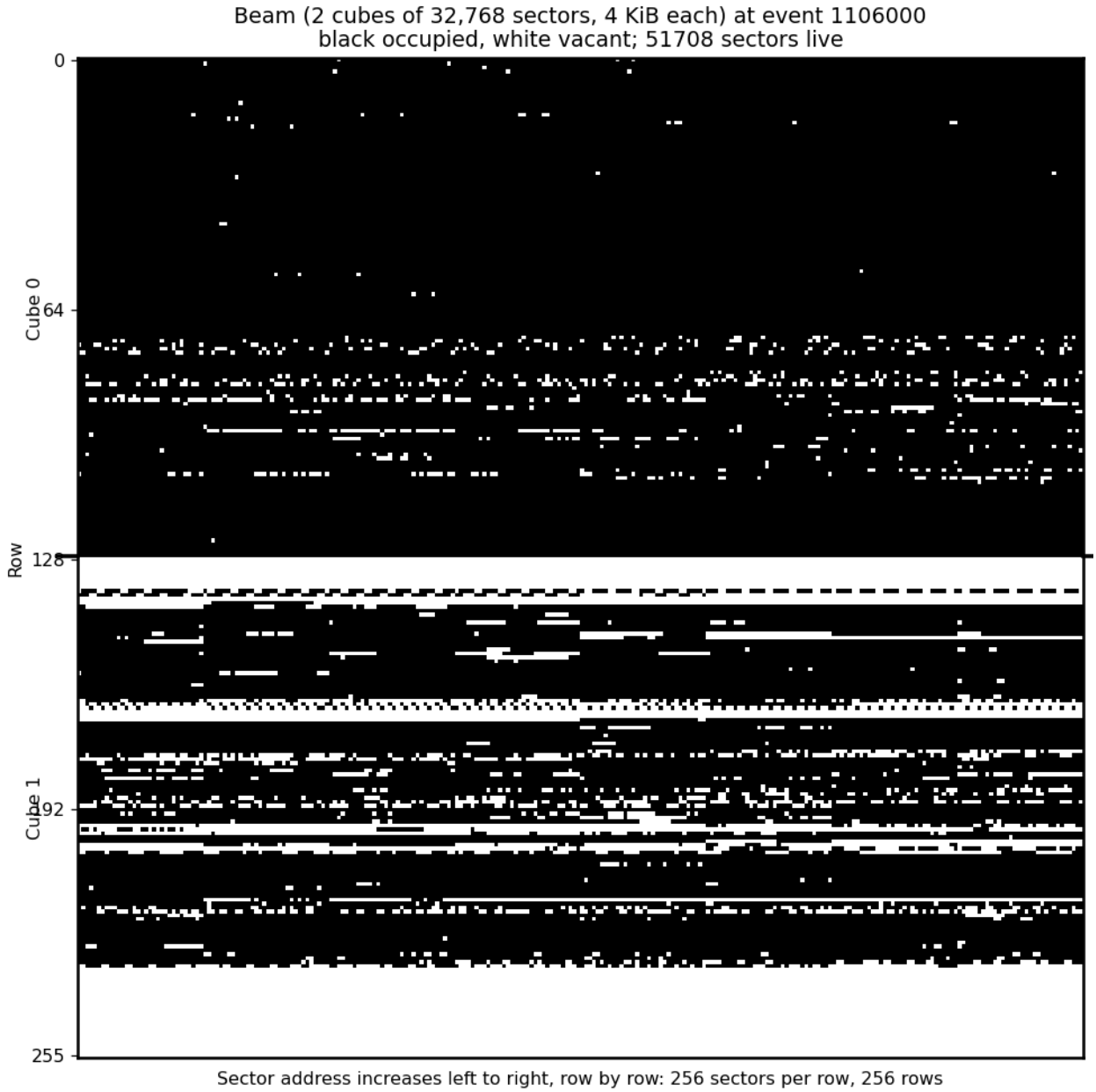


Figure 6: The beam at the top of the crest, two cubes, 256 MiB arena, event 1,106,000. Black is occupied, white is vacant; each cell is one 4 KiB sector. Cube 0 stands 97.5 percent full: 809 sectors free in 503 runs, its largest run 15. Cube 1 holds the reserve: 13,019 sectors free, its largest run 5,920.

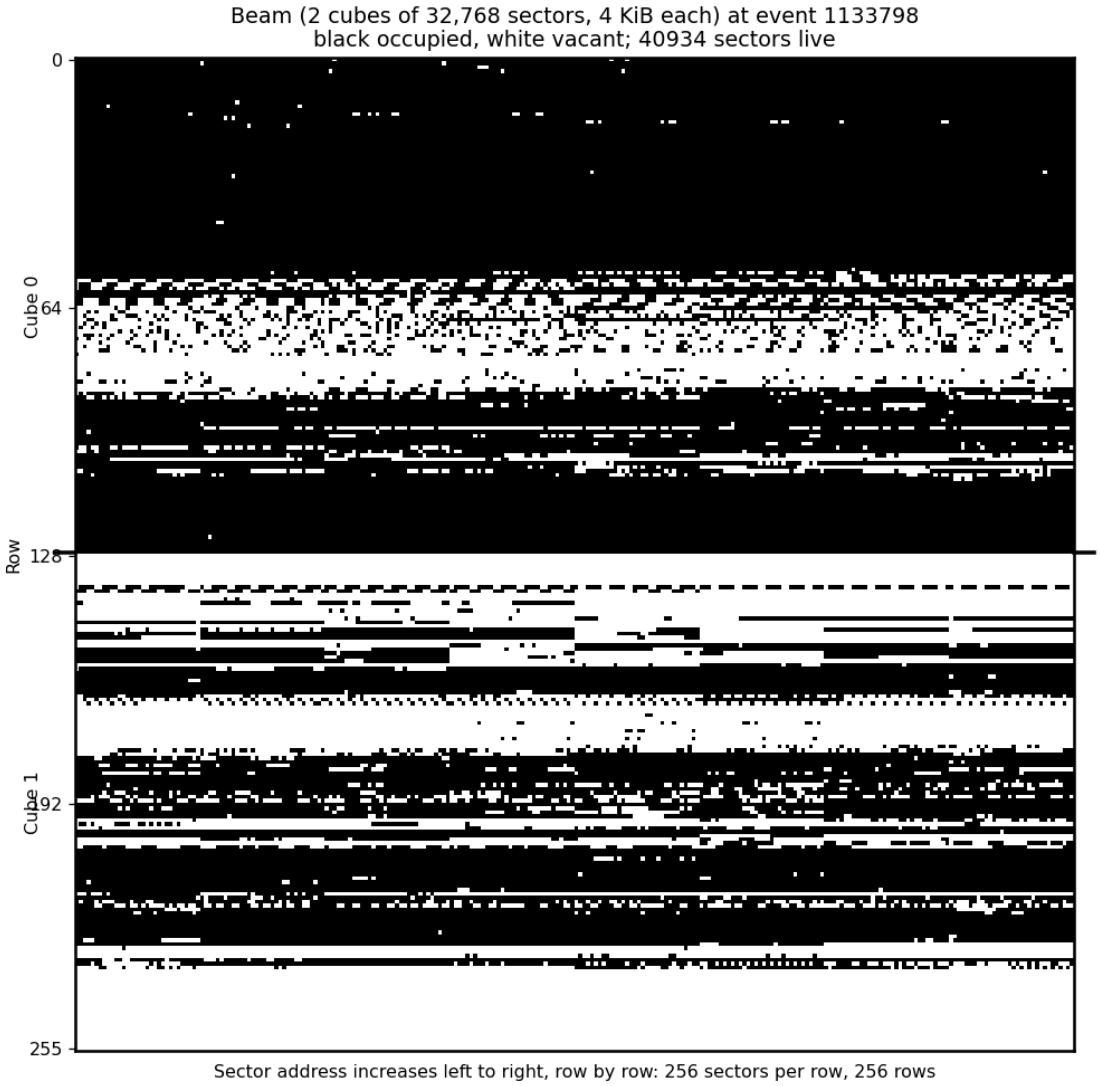


Figure 7: The beam at the bottom of the fall, two cubes, 256 MiB arena, event 1,133,798. Black is occupied, white is vacant; each cell is one 4 KiB sector. 24,602 sectors stand free. Cube 0's largest run stands restored at 970 sectors, and cube 1's at 5,377.

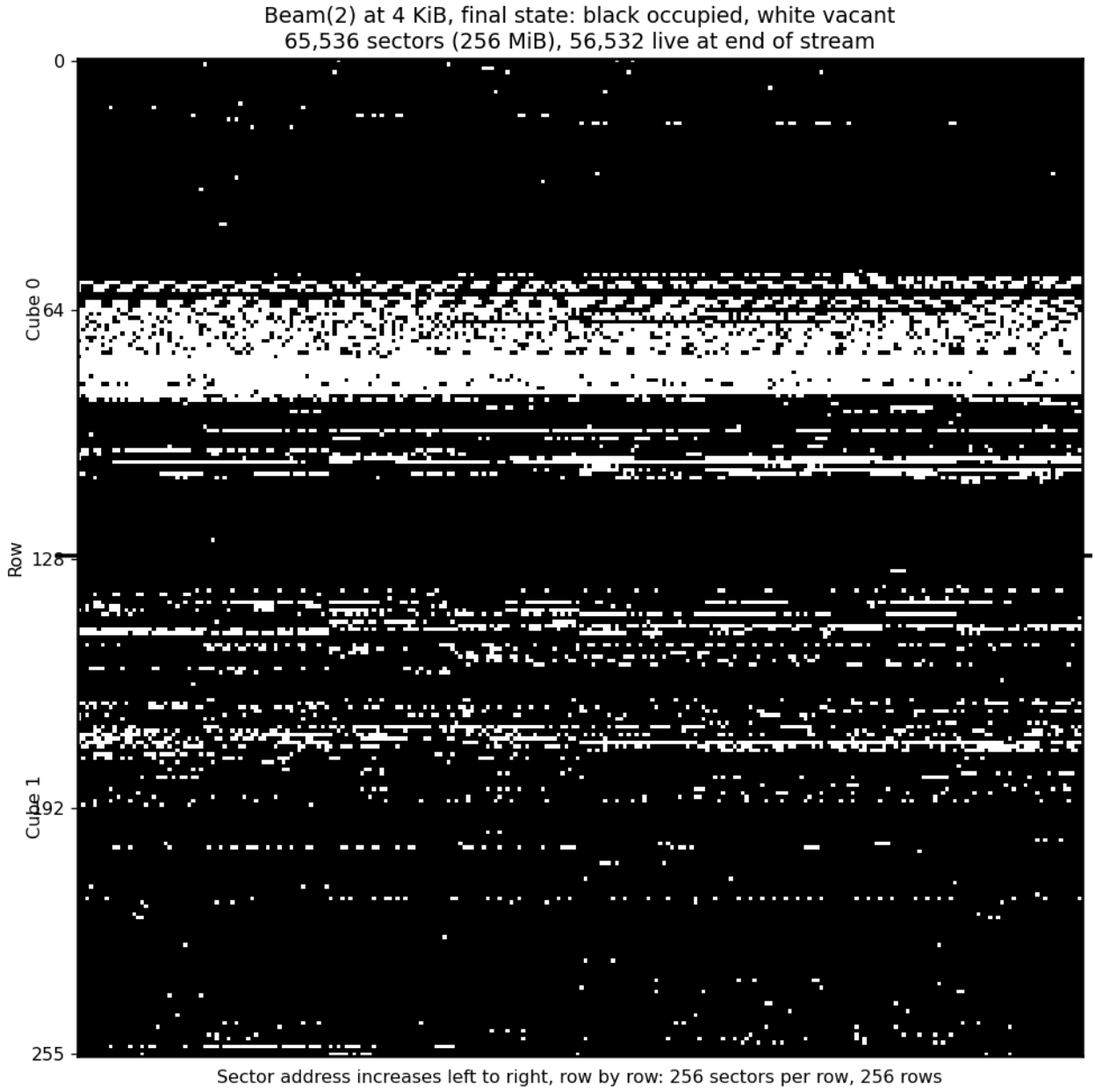


Figure 8: The beam at the end of the stream, two cubes, 256 MiB arena. Black is occupied, white is vacant; each cell is one 4 KiB sector. 56,532 sectors stand live and 9,004 free, in a fine speckle of runs across both cubes, every vacant sector reusable.

9 Conclusion

The beam's maps look scattered at every instant, and the scatter is the design rather than a symptom. Because placement is exact and guided by the oracles, a freed sector is indistinguishable from one never granted: there is no coalescing step, no buddy that must also die before a free sector can be spent again. The transfer of free sectors back into service is nearly instant. Between the crest and the bottom, 10,774 sectors passed from live to free, and every one of them was available to the very next request at its own size. The buddy system's gathered pool is bought with work on every free, its merge cascade run in anticipation of large usages that may never arrive.

The beam does no such work and anticipates nothing: under high fragmentation it spends the runs that stand, at the sizes they stand. The cost of the design is the one the maps also show: a fit is walled inside a single cube, so the beam's largest single fit is whatever the emptiest cube can offer whole. What the beam keeps, at every instant, is the whole of its free space in spendable form.

10 Questions & Answers

How are free adjacent sectors joined? They are not joined through a process. They are recognized as adjacent during an O(1) search via the oracles. There is no coalescing pass and nothing is maintained on the free path: contiguity is a fact the tables already know, not a structure the allocator builds. The details are in the author's first paper, Project Iscandar [1].

The arena stood mostly empty, so what does the test prove? How the arena stood empty matters more than that it stood empty. Two systems built for two entirely different problem classes behaved identically under the same load, with a slight nod toward Iscandar on overall savings: 56,535 sectors against 56,599 pages at the peak.

Is a mathematical model enough, without running the allocator in a live system? Mathematical modelling to stringent enough parameters is sufficient for this test. The next step is to directly patch Fubuki's operating system, and this is left as an exercise for the hacker.

Is this not a special-purpose allocator tested outside its element? Iscandar is designed to allocate from a closed memory system. That it can provide working memory under conventional conditions at all is unusual.

Are the oracle tables not too heavy to be practical? The oracles need only be generated once per microprocessor, and they can be shared between Cubes, the fundamental large scale allocator unit. One generated set serves every cube in the system; it is not generated per cube, and the whole of it fits on a single thumb drive.

Acknowledgements

The author is increasingly disabled from diabetes. At his direction, Muse translated his theorems into Python code, ran tests for him, and created the graphs in this paper from those tests.

References

[1] Andrew Jonathan Fine. An extension of buddy systems to arbitrary radix for deterministic memory allocation. Monograph (Iscandar 45), Zenodo, 2026. Software and archive: github.com/IronAutie/iscandar, release v1.0.0.

[2] Donald E. Knuth. *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*. Addison-Wesley, 2 edition, 1973. Section 2.5, Dynamic Storage Allocation; 1st ed. 1968.

[3] J. M. Robson. Worst case fragmentation of first fit and best fit storage allocation strategies. *The Computer Journal*, 20(3), 1977.

[4] Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles. Dynamic storage allocation: A survey and critical review. In *Proc. Int'l Workshop on Memory Management (IWMM)*, volume 986 of *LNCS*, pages 1–116. Springer, 1995.